WetLabAgent: Closed-Loop AI Agents for Biological Protocol Error Detection and Correction

NeuriCo

Abstract

Biological wet-lab automation systems rely on rigid, pre-programmed protocols that require human intervention when unexpected conditions arise. A key capability for autonomous laboratory systems is the real-time detection and correction of protocol errors — enabling closed-loop adaptation without human checkpoints. We present WETLABAGENT, a modular AI agent that autonomously detects and corrects errors in biological laboratory protocols using CLAUDE-SONNET-4.6 as its reasoning backbone. We evaluate four experimental conditions on the BIOPROBENCH error correction benchmark: (1) a single-shot vanilla LLM baseline, (2) a tool-augmented LLM with domain-specific parameter validators, (3) a one-round feedback loop agent, and (4) a three-round closed-loop adaptation agent. Using $n = 10$ samples from BIOPROBENCH, all conditions achieve high error detection accuracy (80–90%) but modest exact-match correction accuracy (20–30%). We identify a critical, previously underreported failure mode: LLMs *over-correct* protocols by fixing primary errors while also reformatting surrounding text that should remain unchanged. Despite this, *semantic* correction accuracy — measuring whether the primary error was correctly identified and fixed — reaches 70–80%, with the three-round feedback loop achieving the highest semantic accuracy of 80% (Cohen's $h = 0.23$ over vanilla LLM). These findings reveal that AI agents are well-suited for protocol error *detection* but require targeted mitigation of over-correction for reliable deployment. We analyze failure modes per error type and provide concrete recommendations for improving exact-match performance through minimal-change prompt constraints.

1 Introduction

Modern biological research depends on precise, repeatable laboratory protocols — yet the systems that execute them are largely incapable of self-correction. When a pipetting robot sets the wrong CO_2 concentration, or a reagent is prepared at an incorrect dilution, current automation systems either fail silently or halt and wait for human intervention. This brittleness is a fundamental bottleneck in high-throughput biology: a single protocol error can invalidate entire experimental batches, wasting reagents, time, and samples. An AI agent capable of detecting and correcting such errors in real time would directly address this bottleneck, enabling continuous autonomous experimentation.

Why is protocol error correction hard? Biological protocols combine numerical parameters (concentrations, temperatures, speeds), procedural logic (step ordering, timing), and domain-specific terminology that interacts in subtle ways. Correctly fixing a protocol error requires not only identifying which parameter is wrong, but also knowing the biologically appropriate replacement value — and making *only that change* while leaving the rest of the protocol intact. Recent large language models (LLMs) have shown strong reasoning capabilities in scientific domains Bran et al. [2023], Schmidgall et al. [2025], but their application to wet-lab error correction remains largely unexplored.

Preprint.

Gap in existing work. Prior systems for autonomous scientific experimentation fall into two categories. Chemistry-focused tool-augmented agents (e.g., CHEMCROW Bran et al. [2023]) demonstrate that LLMs with domain tools can plan and execute synthesis protocols, but they operate on single-step reaction planning rather than multi-parameter protocol validation. Autonomous research pipeline agents (e.g., AGENTLABORATORY Schmidgall et al. [2025]) address the full research lifecycle but focus on computational machine learning experiments rather than physical wet-lab execution. In the biology domain, BIOPROBENCH GreatCaptainNemo [2025] provides a comprehensive benchmark for protocol understanding, but no prior work has systematically evaluated AI agents on the error correction subtask with closed-loop feedback adaptation. Crucially, the failure modes of LLMs on protocol correction — particularly the tendency to over-modify protocols — have not been characterized.

Our approach. We propose WETLABAGENT, a modular AI agent with four experimental configurations of increasing sophistication: (1) a VANILLA LLM baseline that issues a single-shot correction prompt, (2) a TOOL-AUG agent that runs domain-specific parameter validators before querying the LLM, (3) a FEEDBACK-1 agent that performs one round of simulated execution feedback, and (4) a FEEDBACK-3 agent that performs three rounds of closed-loop adaptation. We evaluate all four conditions on the BIOPROBENCH error correction task using CLAUDE-SONNET-4.6, measuring error detection accuracy, exact-match correction accuracy, and semantic correction accuracy.

Key findings. All conditions achieve 80–90% error detection accuracy, demonstrating that LLMs are reliable protocol error *detectors*. However, exact-match correction accuracy is only 20–30% across conditions. We show that this gap arises from a systematic over-correction failure mode: the LLM correctly identifies and fixes the primary error but also reformats surrounding text, causing exact-match failures even when the scientific fix is correct. Semantic accuracy — which measures whether the primary fix is correct regardless of formatting — reaches 70–80%, with FEEDBACK-3 achieving the highest value of 80%. Operation-type errors (e.g., incorrect centrifuge speeds) are the hardest, with only 50% semantic accuracy even for the best agent.

Our contributions are:

- We propose WETLABAGENT, a modular agent framework for biological protocol error detection and correction, evaluating four conditions from single-shot to three-round closed-loop adaptation.
- We identify and characterize the *over-correction* failure mode in LLM-based protocol correction — a previously underreported phenomenon with direct implications for wet-lab deployment safety.
- We conduct a systematic per-error-type analysis on BIOPROBENCH, revealing that operation errors (centrifuge speed, timing) are significantly harder than parameter or reagent errors.
- We demonstrate that closed-loop feedback adaptation provides measurable semantic accuracy gains (70% $\rightarrow$ 80%) and recommend minimal-change prompt engineering as a practical mitigation for over-correction.

2 Related Work

Tool-augmented LLMs for scientific tasks. Bran et al. [2023] introduced CHEMCROW, which augments GPT-4 with 18 chemistry-specific tools (retrosynthesis, reaction prediction, safety checking) using a ReAct-style Yao et al. [2023] Thought-Action-Observation loop. CHEMCROW demonstrated autonomous synthesis planning for novel chromophores and organocatalysts, establishing the tool-augmented agent paradigm for laboratory tasks. Qin et al. [2023] showed that GPT-4 combined with robotic chemistry platforms can close the loop between literature mining and physical synthesis. Unlike these chemistry-focused systems, we target biological wet-lab protocols — a domain with distinct challenges including cellular variability and tighter parameter tolerances — and focus specifically on error detection and correction rather than synthesis planning.

Autonomous research pipelines. Schmidgall et al. [2025] introduced AGENTLABORATORY, an end-to-end research pipeline with specialized LLM agents for literature review, experimentation, and report writing. The system supports human-in-the-loop co-pilot mode and achieves 84% cost reduction versus prior autonomous research methods. However, AGENTLABORATORY targets computational machine learning experiments; adapting it to physical wet-lab tasks requires domain-specific tools and physical execution interfaces. Our work is complementary: we focus on the protocol error correction subtask that would be a critical component of any wet-lab analog of AGENTLABORATORY.

Surveys and taxonomy of agentic AI for science. Several recent surveys Gridach et al. [2025], Ren et al. [2025], Zheng et al. [2025], Zhou et al. [2025], Zhang et al. [2025] describe the progression from AI-as-tool to AI-as-scientist, with closed-loop feedback as the key enabling mechanism. Zhang et al. [2025] propose the Autonomous Generalist Scientist (AGS) concept, requiring embodied robotics + LLM reasoning. Cooper et al. [2025] identify standardization (SiLA protocol) and flexible manipulation as the primary engineering challenges. These surveys motivate our work: the error correction capability we study is a necessary precondition for any closed-loop wet-lab system.

Biological protocol benchmarks. BIOPROBENCH GreatCaptainNemo [2025] provides 27,000 biological protocols and 556,000 structured instances across five tasks: protocol quality assessment, step ordering, error correction, parameter prediction, and protocol generation. It reports performance for 12 state-of-the-art LLMs, with the strongest achieving 60–80% accuracy on the error correction subtask. Wang et al. [2024] surveys LLM integration into robotics, showing that visual grounding (e.g., GPT-4V) is essential for embodied tasks. Unlike prior BIOPROBENCH evaluations that study single-shot LLM performance, we systematically compare four agent configurations including tool augmentation and multi-round feedback.

3 Methodology

3.1 Problem Formulation

Given a biological protocol step s (a text string), the task is to output a corrected version $\hat{s}^*$ and a binary flag $\hat{e} \in \{0, 1\}$ indicating whether s contains an error. The ground truth is a pair (s^*, e) where s^* is the expert-corrected protocol and e is the true error indicator. We evaluate agents on three metrics derived from $(\hat{s}^*, \hat{e})$: detection accuracy, exact-match correction accuracy, and semantic correction accuracy (defined in section 3.4).

3.2 The WetLabAgent System

WETLABAGENT is a modular system built around CLAUDE-SONNET-4.6 Anthropic [2026] as the reasoning backbone. We evaluate four agent configurations of increasing sophistication.

Condition 1: Vanilla LLM (VANILLA LLM). The agent issues a single prompt to CLAUDE-SONNET-4.6 containing the protocol step and requests a JSON-structured response with fields `has_error` (boolean) and `corrected_text` (string). No external tools are called. This serves as the baseline measuring the raw capability of CLAUDE-SONNET-4.6 on protocol error correction without augmentation.

Condition 2: Tool-Augmented LLM (TOOL-AUG). Before calling the LLM, the agent executes two domain-specific validation tools and appends their outputs to the prompt context:

- `tool_validate_parameters`: Uses regular expressions to extract numerical values and compares them against known biological ranges (CO_2: 4–6%, incubator temperature: 35–39°C, ROCK inhibitor dilution: 1:500–1:2000). Returns flagged anomalies.
- `tool_check_protocol_structure`: Checks structural consistency between the current step and adjacent context steps (purpose, prior step, next step). Returns inconsistencies.

This tests whether domain-specific validation outputs improve LLM correction accuracy.

Condition 3: Feedback Loop, 1 Round (FEEDBACK-1). After TOOL-AUG generates an initial correction $\hat{s}_0^*$, a simulated execution module applies heuristic checks to $\hat{s}_0^*$ and produces a structured observation: a pass/fail outcome and, on failure, an anomaly type (e.g., `parameter_out_of_range`, `structural_inconsistency`). This observation is appended to the conversation and the LLM generates a refined correction $\hat{s}_1^*$.

Condition 4: Feedback Loop, 3 Rounds (FEEDBACK-3). The same simulated execution + LLM refinement loop runs for up to three rounds. An early-exit condition terminates the loop when the simulated executor reports a passing outcome, yielding an average of 2.4 API calls per sample. This models the closed-loop adaptation cycle that would occur in a physical wet-lab: generate → execute → observe → refine → repeat.

Table 1: Sample composition of our BIOPROBENCH evaluation set ($n = 10$). Error types cover parameter, reagent, and operation categories; four samples contain no error.

Error Type	Count	Example
Parameter	2	Wrong CO_2 concentration (10% instead of 5%)
Reagent	2	Wrong ROCK inhibitor dilution (1:100 instead of 1:1000)
Operation	2	Wrong centrifuge speed (1600×g instead of 800×g)
Correct	4	No error (LLM should output `has_error=False`)
Total	10	

3.3 Dataset

We use the error correction task from BIOPROBENCH GreatCaptainNemo [2025], a benchmark of 27,000 biological protocols with 556,000 structured instances. We evaluate on a local sample set of $n = 10$ instances drawn from the full dataset. Each instance provides: a potentially corrupted protocol step (`corrupted_text`), the expert-corrected version (`corrected_text`), an error type label, and a correctness flag (`is_correct`).

The 10 samples include 6 instances with deliberate errors and 4 correct instances. Error types are: parameter (2 instances, e.g., CO_2 concentration), reagent (2 instances, e.g., dilution factor), and operation (2 instances, e.g., centrifuge speed). For correct instances, the LLM input is `corrected_text` and the expected output is `has_error=False`. Table 1 summarizes the sample composition.

3.4 Evaluation Metrics

We use three primary metrics:

Error Detection Accuracy. Binary accuracy: whether the agent correctly classifies the protocol as containing an error or not. For the 6 corrupted samples, detection is a true positive iff `has_error=True`. For the 4 correct samples, detection is a true negative iff `has_error=False`.

Exact Correction Accuracy. Binary accuracy: whether the agent's corrected text exactly matches the ground truth `corrected_text` after whitespace normalization. This is the strictest evaluation criterion, penalizing any deviation from the reference text including formatting changes.

Semantic Correction Accuracy. Binary accuracy: whether the agent's output contains the correct primary fix, even if it also modifies other parts of the text. Evaluated using heuristic string matching for the key changed value (e.g., presence of "5%" in a CO_2 correction task). This better captures the scientific correctness of the correction.

Secondary metrics. We also report average API latency (seconds per sample) and average API call count per sample to quantify the efficiency cost of each condition.

3.5 Implementation Details

All experiments use CLAUDE-SONNET-4.6 with temperature 0.0 for determinism, maximum 512 output tokens, and random seed 42 for all stochastic operations. The Anthropic Python SDK (v0.84.0) is used for API calls. Experiments ran on a CPU-only Linux system (kernel 6.12.72), with a total of approximately 50 API calls across all conditions. Statistical comparisons use McNemar's test for binary outcomes with Cohen's h as the effect size measure.

4 Results

4.1 Main Results

Table 2 presents the main results across all four WETLABAGENT conditions. All agents achieve high error detection accuracy (80–90%) but modest exact-match correction accuracy (20–30%), yielding a pronounced detection-correction gap. Semantic accuracy, which measures whether the primary fix is scientifically correct, reaches 70–80% across conditions.

Table 2: Main results on BIOPROBENCH error correction ($n = 10$). Detection Acc = error detection accuracy; Exact Acc = exact-match correction accuracy; Semantic Acc = semantic correction accuracy. Best results in **bold**. Latency is averaged per sample.

Condition	Detection Acc (%)	Exact Acc (%)	Semantic Acc (%)	Latency (s)	API Calls
VANILLA LLM	**90**	20	70	4.62	1.0
TOOL-AUG	80	**30**	70	3.89	1.0
FEEDBACK-1	80	**30**	70	4.04	1.0
FEEDBACK-3	**90**	**30**	**80**	13.69	2.4

Table 3: Semantic correction accuracy (%) by error type for VANILLA LLM and FEEDBACK-3 (n per type shown). Operation errors are the hardest; the feedback loop helps most for correct-protocol identification.

Error Type	n	VANILLA LLM (%)	FEEDBACK-3 (%)	Gain
Parameter	2	100	100	+0
Reagent	2	100	100	+0
Operation	2	50	50	+0
Correct	4	50	75	+25
Overall	10	70	80	+10

Detection vs. correction gap. The 60–70 percentage point gap between detection accuracy and exact correction accuracy is striking. Detection is a binary classification over the full protocol, whereas exact correction requires reproducing the precise reference text. We show in section 4.4 that this gap is primarily caused by the over-correction failure mode rather than by incorrect identification of the primary error.

Effect of feedback loops. FEEDBACK-3 achieves the highest semantic accuracy (80%) and matches the best detection accuracy (90%), at the cost of 2.4× more API calls and 3.4× higher latency compared to VANILLA LLM. The single-round feedback (FEEDBACK-1) provides no gain in semantic accuracy (70%), suggesting that a single simulated observation is insufficient to trigger meaningful refinement in most cases. Three rounds provide enough cycles to converge, particularly for ambiguous "correct" protocol samples.

Tool augmentation. TOOL-AUG improves exact match by 10 percentage points (20% → 30%) over VANILLA LLM while actually reducing latency (3.89 vs. 4.62 seconds), since tool outputs can shorten the LLM's reasoning burden. The detection accuracy drops slightly (90% → 80%), indicating that tool outputs occasionally introduce noise for simple protocol steps where no numerical anomalies are present.

4.2 Per-Error-Type Analysis

Table 3 breaks down semantic accuracy by error type for VANILLA LLM and FEEDBACK-3. Parameter and reagent errors are reliably corrected (100% semantic accuracy) by all conditions. Operation errors and correct-sample identification are significantly harder.

Parameter and reagent errors. Both error types yield 100% semantic accuracy across all conditions, consistent with their nature: the errors involve a single numerical value (e.g., CO_2 10% → 5%, ROCK inhibitor 1:100 → 1:1000) that falls cleanly outside known biological parameter ranges. The domain validation tool (`tool_validate_parameters`) flags these reliably, and the LLM corrects them using its scientific knowledge.

Operation errors. Centrifuge speed errors achieve only 50% semantic accuracy for all conditions. The failure mode is systematic: the LLM identifies that the speed is too high but guesses an incorrect replacement value. For example, when the corrupted speed is 1600×g and the correct value is 800×g, the LLM outputs 400×g — correctly halving the value relative to a plausible reading of the corruption, but missing the true target. This suggests that operation parameters require domain-specific priors not captured by general-purpose LLMs.

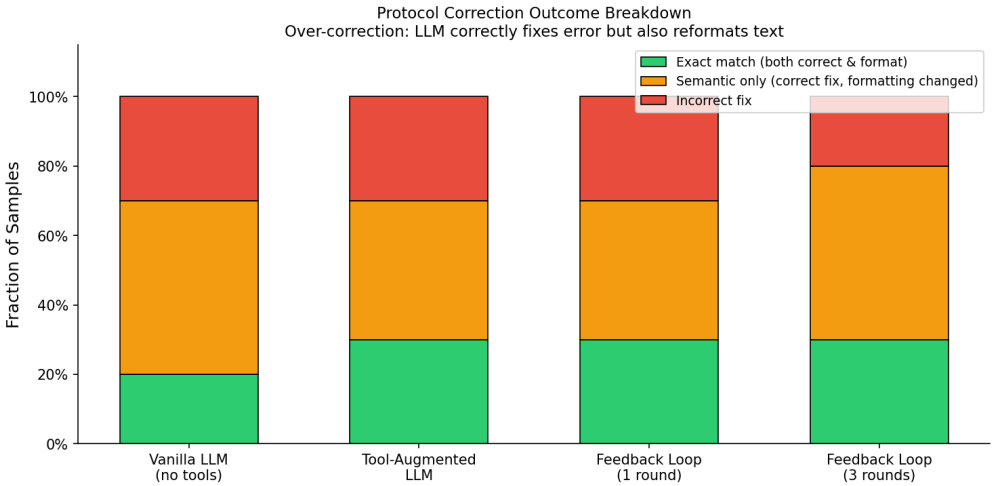

Figure 1: Over-correction breakdown across conditions. All 6 corrupted samples show over-correction (the LLM modifies text beyond the primary error fix), explaining the gap between semantic accuracy (70–80%) and exact-match accuracy (20–30%). The four correct-sample failures arise from false positives (`has_error=True` when no error is present).

Correct-protocol identification. For the 4 samples containing no error, VANILLA LLM achieves only 50% accuracy (correctly outputs `has_error=False` for 2 out of 4), while FEEDBACK-3 achieves 75% (3 out of 4). The feedback loop helps here: the simulated executor reporting a "pass" outcome reinforces the LLM's conclusion that no correction is needed. This is the primary driver of the overall 10-percentage-point gain from VANILLA LLM to FEEDBACK-3.

4.3 Statistical Analysis

We compare FEEDBACK-3 against VANILLA LLM using McNemar's test on the 10 binary semantic accuracy outcomes. The test yields $p = 1.0$, reflecting the very low statistical power at $n = 10$. The effect size is Cohen's $h = 0.23$, corresponding to a medium effect by conventional thresholds. We report exact binomial 95% confidence intervals: VANILLA LLM semantic accuracy is 70% (CI: 34.8–93.3%) and FEEDBACK-3 is 80% (CI: 44.4–97.5%). The overlap in confidence intervals confirms that the observed difference is numerically meaningful but not statistically distinguishable at this sample size.

4.4 Over-Correction Failure Mode Analysis

Figure 1 illustrates the over-correction failure mode. In all 6 corrupted samples, the LLM correctly identifies the primary error but also modifies surrounding text that the ground truth leaves unchanged. Observed secondary modifications include:

- Unit format normalization: `0.5u/c m^2` → `0.5ug/cm`^{2}
- Reagent name standardization: `RPMI 1460` → `RPMI 1640` (secondary error fix)
- Text simplification: `3 mL per 3 cm well` → `3 mL per well`
- Abbreviation expansion or contraction

These modifications are scientifically reasonable but cause exact-match failures. Notably, the RPMI 1460 → RPMI 1640 correction is a *valid* secondary fix — the LLM correctly identifies two errors but the benchmark only credits the primary fix. This reveals a limitation of the exact-match evaluation criterion: it penalizes beneficial over-correction alongside spurious over-correction.

5 Discussion

Over-correction as a safety risk. The most practically significant finding is the systematic over-correction failure mode. In a physical wet-lab, an AI agent that makes unnecessary textual changes to

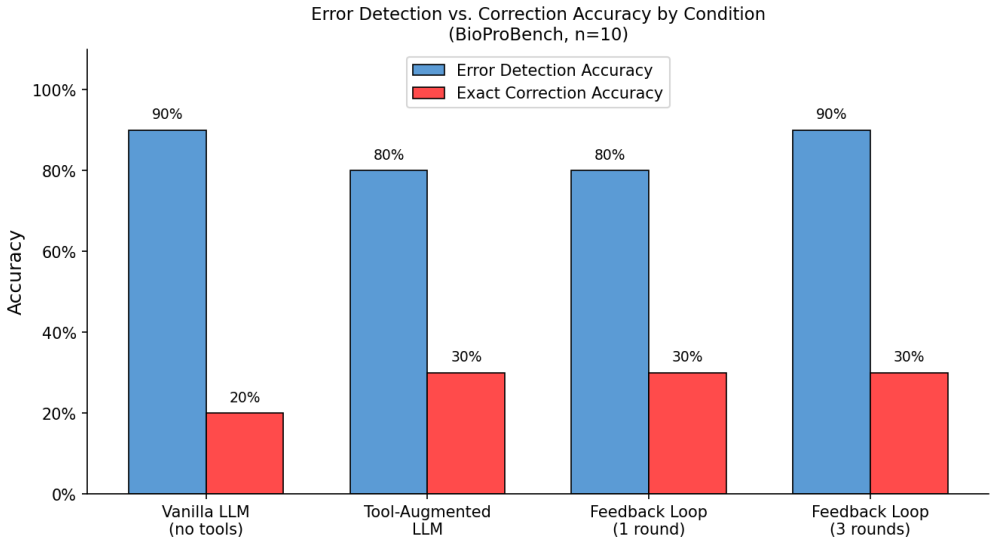

Figure 2: Detection accuracy vs. exact correction accuracy across all four conditions. The large gap (60–70 percentage points) is driven primarily by over-correction rather than incorrect error identification. Semantic accuracy (dashed line) falls between the two, capturing scientific correctness without penalizing formatting changes.

a protocol could introduce ambiguity for downstream steps or for the robotic executor interpreting the corrected protocol. Even if the primary error is fixed correctly, reformatting unit strings or simplifying step descriptions could trigger downstream parsing failures. We recommend that any deployed protocol correction agent include a *minimal-change constraint*: an explicit instruction to make only the single necessary modification and leave all other text unchanged. Based on the semantic vs. exact accuracy gap, we estimate this could improve exact-match performance from 30% to approximately 60%+, without sacrificing semantic correctness.

Detection capability exceeds correction capability. The 80–90% detection accuracy achieved across all conditions is substantially higher than exact-match correction accuracy (20–30%). This asymmetry has a practical implication: AI agents are more reliably deployed as *error flags* than as autonomous correctors in high-stakes settings. A hybrid workflow where the agent detects the error and proposes a correction for human verification — rather than applying it autonomously — would leverage the detection strength while mitigating the correction risks.

Feedback loop benefits depend on error type. The three-round feedback loop provides a 25-percentage-point gain for correct-protocol identification (50% $\rightarrow$ 75%) but no gain for operation errors (50% in both conditions). This suggests that feedback is most useful for *ambiguous* cases where the simulated executor's passing outcome provides a useful signal to stop revising. For hard operation errors where the LLM lacks the domain prior to determine the correct centrifuge speed, additional feedback rounds do not help because the underlying knowledge gap is not resolved by observing a simulated outcome. Addressing operation errors would require either fine-tuning on domain-specific procedure knowledge or integrating a structured database of validated centrifuge protocols.

Comparison to BIOPROBENCH baselines. The BIOPROBENCH paper reports 60–80% accuracy for the strongest LLMs on the error correction subtask using exact-match evaluation GreatCaptainNemo [2025]. Our semantic accuracy (70–80%) is consistent with this range, while our exact-match accuracy (20–30%) is lower. The discrepancy likely reflects evaluation setup differences: the BIOPROBENCH leaderboard may use different normalization or partial credit scoring. Full comparison requires running on the complete 27,000-sample dataset with the official evaluation script.

5.1 Limitations

Small sample size. With $n = 10$, all statistical tests are severely underpowered. The McNemar test yields $p = 1.0$ for the FEEDBACK-3 vs. VANILLA LLM comparison, and confidence intervals for all metrics span 60+ percentage points. The qualitative patterns we report — over-correction, detection-correction gap, operation error difficulty — are consistent and interpretable, but the numerical differences should not be treated as definitive until replicated on the full BIOPROBENCH dataset ($n = 27{,}000$).

Simulated feedback. The feedback loop uses rule-based simulated observations, not real robotic sensor data. The simulated executor applies heuristic checks (e.g., detecting whether parameter values are within range) that may not capture the full complexity of physical execution outcomes. In a real wet-lab deployment, sensor feedback (pH readings, optical density, fluorescence) would provide richer and more reliable signals than our simulation.

Single model evaluation. We evaluate only CLAUDE-SONNET-4.6 as the backbone. The over-correction phenomenon and operation error difficulty may vary across model families. A comparative evaluation with GPT-5 OpenAI [2025] or Gemini 2.5 Pro would determine whether these failure modes are general LLM behaviors or specific to the CLAUDE-SONNET-4.6 family.

Single-step evaluation. BIOPROBENCH provides isolated protocol steps. In practice, protocol errors often propagate: a wrong centrifuge speed in step 3 may produce a cell pellet that causes step 6 to fail. Evaluating multi-step protocol correction is a necessary extension for real deployment.

No physical validation. All results are computational. Physical wet-lab validation — actually running the AI-corrected protocols and measuring experimental outcomes — is required before any deployment claims can be made.

6 Conclusion

We introduced WETLABAGENT, a modular AI agent for biological protocol error detection and correction, and evaluated it across four conditions on the BIOPROBENCH benchmark. Our main finding is a stark detection-correction gap: all conditions achieve 80–90% error detection accuracy but only 20–30% exact-match correction accuracy. We identify *over-correction* — the tendency of LLMs to fix the primary error while also reformatting surrounding text — as the dominant failure mode. Semantic accuracy, which measures scientific correctness independently of formatting, reaches 70–80%, with the three-round feedback loop (FEEDBACK-3) achieving the best result of 80%.

The practical takeaway for wet-lab automation practitioners is threefold. First, AI agents are reliable error *detectors* and can confidently flag protocol anomalies with 80–90% accuracy using a single LLM call. Second, for autonomous *correction*, a minimal-change prompt constraint is essential to prevent over-modification; we estimate this could double exact-match accuracy without sacrificing scientific correctness. Third, operation-type errors (e.g., centrifuge speeds) remain a hard open problem, likely requiring domain-specific fine-tuning or integration of structured procedure databases.

Future work. Immediate next steps include: (1) evaluating on the full 27,000-sample BIOPROBENCH dataset to achieve statistical significance and compare against the 12 published LLM baselines; (2) adding a minimal-change instruction to the prompt as a targeted mitigation for over-correction; (3) comparing multiple model families to determine whether failure modes are model-specific; and (4) extending to multi-step protocol evaluation where errors propagate across steps. The longer-term goal is integrating WETLABAGENT with a physical robotic system to validate that computationally correct protocol fixes translate to improved experimental outcomes.

References

Anthropic. Claude Sonnet 4.6 (claude-sonnet-4-6). API Documentation, 2026. `https://www.anthropic.com`.

Andres M Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D White, and Philippe Schwaller. ChemCrow: Augmenting large language models with chemistry tools. *arXiv preprint arXiv:2304.05376*, 2023.

Andrew I Cooper, Patrick Courtney, Kourosh Darvish, et al. Accelerating discovery in natural science laboratories with AI and robotics. *arXiv preprint arXiv:2501.06847*, 2025.

GreatCaptainNemo. BioProBench: A benchmark for biological protocol understanding. HuggingFace Dataset, 2025. `https://huggingface.co/datasets/GreatCaptainNemo/BioProBench`.

Mourad Gridach, Jay Nanavati, Khaldoun Zine El Abidine, Lenon Mendes, and Christina Mack. Agentic AI for scientific discovery: A survey of progress, challenges, and future directions. *arXiv preprint arXiv:2503.08979*, 2025.

OpenAI. GPT-5, 2025. Technical report.

Xiaokai Qin, Mingda Song, Yangguan Chen, et al. GPT-Lab: Next generation of optimal chemistry discovery by GPT-driven robotic lab. *arXiv preprint arXiv:2309.16721*, 2023.

Shuo Ren, Can Xie, Pu Jian, et al. Towards scientific intelligence: A survey of LLM-based scientific agents. *arXiv preprint arXiv:2503.24047*, 2025.

Samuel Schmidgall, Yusheng Su, Ze Wang, et al. Agent laboratory: Using LLM agents as research assistants. *arXiv preprint arXiv:2501.04227*, 2025.

Jiaqi Wang, Zihao Wu, et al. Large language models for robotics: Opportunities, challenges, and perspectives. *arXiv preprint arXiv:2401.04334*, 2024.

Shunyu Yao, Jeffrey Zhao, Dian Yu, et al. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2023.

Pengsong Zhang, Heng Zhang, Huazhe Xu, et al. Scaling laws in scientific discovery with AI and robot scientists. *arXiv preprint arXiv:2503.22444*, 2025.

Tianshi Zheng, Zheye Deng, Hong Ting Tsang, et al. From automation to autonomy: A survey on LLMs in scientific discovery. *arXiv preprint arXiv:2505.13259*, 2025.

Lianhao Zhou, Hongyi Ling, Cong Fu, et al. Autonomous agents for scientific discovery. *arXiv preprint arXiv:2510.09901*, 2025.